

72.07 Protocolos de Comunicación

Trabajo Práctico Especial



Servidor Proxy SOCKS5 y Protocolo de Monitoreo

Integrantes Grupo 19:

- Francisco Costa, 65202
- Roman Salerno, 65145
- Franco Branda, 65506

Fecha de Entrega: 14/07/2026

Índice

1 - Resumen.....	2
2 - Protocolos y aplicaciones desarrolladas.....	2
2.1 Protocolo socks5.....	2
2.2 Protocolo de monitoreo.....	3
2.3 Aplicación cliente.....	4
3 - Problemas encontrados y proceso de desarrollo.....	4
3.1 Proceso de desarrollo.....	4
3.2 Roles.....	5
3.3 Problemas encontrados.....	5
4 - Limitaciones de la aplicación.....	6
5 - Posibles extensiones.....	6
6 - Conclusiones.....	7
7 - Ejemplos de prueba.....	7
8 - Guia de instalación.....	9
9 -Instrucciones para la configuración.....	9
10 - Ejemplos de configuración y monitoreo.....	10
11 - Documento del diseño del proyecto.....	11

1 - Resumen

El presente informe se centra en el diseño e implementación de un servidor proxy SOCKS5 para la materia Protocolos de Comunicación. El servidor atiende múltiples clientes de forma concurrente sobre un único hilo, mediante entrada/salida no bloqueante multiplexada, con autenticación usuario/contraseña y conexiones salientes a direcciones IPv4, IPv6 y nombres de dominio. Se diseñó además un protocolo propio de monitoreo y configuración (SMP), servido en un puerto independiente dentro del mismo proceso, que permite administrar usuarios, consultar métricas, leer el registro de accesos y cambiar la configuración en tiempo de ejecución sin reiniciar el servidor, junto con una aplicación cliente de línea de comandos para operarlo. El documento describe los protocolos y aplicaciones desarrollados, las decisiones de diseño, los problemas y limitaciones encontrados, y presenta ejemplos de prueba - incluyendo pruebas de estrés de concurrencia y throughput - que evidencian su correcto funcionamiento.

2 - Protocolos y aplicaciones desarrolladas

2.1 Protocolo socks5

El servidor desarrollado implementa un proxy SOCKSv5 siguiendo el funcionamiento definido por el RFC 1928. Al iniciarse, crea un socket pasivo para aceptar conexiones entrantes de clientes en el puerto configurado para SOCKS. Por defecto, este puerto es el 1080, aunque puede modificarse al ejecutar el programa.

Para cada cliente que se conecta, el servidor realiza primero la negociación inicial del protocolo. En esta etapa, el cliente informa qué métodos de autenticación soporta y el servidor responde cuál utilizará. El proxy puede funcionar con autenticación por usuario y contraseña, siguiendo el RFC 1929, o sin autenticación, dependiendo de la configuración activa. En caso de utilizar credenciales, estas se validan contra los usuarios cargados al iniciar el servidor o agregados posteriormente desde el protocolo de monitoreo.

Una vez finalizada la negociación, el cliente envía el pedido de conexión indicando el destino al que quiere acceder. El servidor soporta conexiones hacia direcciones IPv4, IPv6 y nombres de dominio. Cuando el destino es un nombre de dominio, se realiza la resolución correspondiente antes de intentar establecer la conexión. Si una dirección falla y existen otras alternativas, el servidor intenta conectarse a la siguiente.

Luego de establecer la conexión con el destino, el servidor pasa a una etapa de reenvío de datos entre el cliente y el servidor remoto. Esto se realiza de forma no bloqueante, utilizando buffers para manejar correctamente lecturas y escrituras parciales. De esta manera, el proxy puede atender múltiples conexiones al mismo

tiempo sin crear un hilo por cliente. Además, se aplica control de flujo: si uno de los extremos recibe datos más lento, el servidor evita acumular memoria sin límite.

Durante el funcionamiento, el proxy registra métricas como cantidad de conexiones, conexiones activas y bytes transferidos. También genera un registro de accesos con información sobre el usuario, el cliente, el destino solicitado y el resultado de la conexión. Esta información luego puede consultarse desde el protocolo de monitoreo implementado en el mismo proyecto.

Cuando una conexión finaliza o ocurre un error, el servidor cierra los sockets involucrados y libera los recursos asociados. Esto permite mantener el servidor funcionando de forma continua, evitando que queden conexiones, memoria o descriptores abiertos innecesariamente.

2.2 Protocolo de monitoreo

El servidor levanta también un servicio de administración en un puerto propio (8080 por defecto), atendido en el mismo proceso y selector no bloqueante que el proxy. Es un protocolo de texto, orientado a línea (cada mensaje termina en CRLF) y sobre TCP; Elegimos TCP porque nos simplificaba mucho la implementación. Para este protocolo necesitábamos mandar comandos y recibir respuestas de forma clara, y con TCP no teníamos que preocuparnos por paquetes perdidos, repetidos o desordenados.

La sesión empieza con un saludo del servidor y el cliente se autentica con un token de administración (distinto de las credenciales del proxy, que son usuario/contraseña):

S: +OK SMP/1.0 ready

C: AUTH <token>

S: +OK authenticated (o **-ERR 4 authentication failed**, y se cierra)

Ya autenticado, el cliente manda comandos de a uno por línea. Cada respuesta empieza con **+OK** si salió bien o con **-ERR <código> <mensaje>** si hubo error. Los comandos disponibles son:

- **GET-METRICS:** métricas del servidor: conexiones históricas, conexiones activas y bytes transferidos.
- **LIST-USERS:** lista los usuarios del proxy.
- **ADD-USER <usuario> <contraseña>:** da de alta un usuario del proxy.
- **DEL-USER <usuario>:** da de baja un usuario del proxy.
- **GET-CONFIG:** muestra la configuración modificable en tiempo de ejecución.
- **SET-CONFIG <clave> <valor>:** cambia una opción de configuración (por ejemplo, exigir o no autenticación en el proxy).

- **GET-LOG:** registro de accesos: quién se conectó, a qué destino y con qué resultado.
- **HELP:** lista los comandos disponibles.
- **QUIT:** cierra la conexión.

Las respuestas que devuelven listas (LIST-USERS, GET-LOG, HELP) ocupan varias líneas y terminan con una línea que contiene únicamente un punto.

2.3 Aplicación cliente

Para no tener que hablar el protocolo a mano, desarrollamos un cliente de línea de comandos que lo abstrae: el usuario nombra una operación y el cliente hace el resto, conectarse, autenticarse con el token, enviar el comando y cerrar.

```
./bin/client 127.0.0.1 8080 -t <token> add-user pablito pass1234
```

La parte de sesión (saludo, autenticación y cierre) no se expone como comandos, sino que la maneja el cliente internamente. La salida se muestra ya interpretada (por ejemplo, las métricas con sus etiquetas), con los datos en la salida estándar, los errores en la de error y código de retorno distinto de cero ante un error. Usa I/O bloqueante y se genera con **make** como **bin/client**.

3 - Problemas encontrados y proceso de desarrollo

Para el desarrollo del proyecto se siguió un enfoque progresivo, intentando tener siempre una versión funcional antes de agregar nuevas funcionalidades. En líneas generales, no se encontraron problemas extremadamente graves ni bloqueos muy complejos. Gran parte del trabajo fue más rápido y claro gracias al uso de herramientas de inteligencia artificial, que ayudaron a entender código de referencia, ordenar alternativas de diseño, detectar errores típicos de programación no bloqueante y acelerar la resolución de dudas durante la implementación.

3.1 Proceso de desarrollo

Al comienzo del proyecto se estudió el código de un repositorio de referencia llamado medias5. Dado que el flujo era completamente secuencial se hizo fácil comprender el funcionamiento del protocolo socks5v, siendo ese nuestro objetivo.

El siguiente paso fue analizar un ejemplo de echo server no bloqueante. Este ejemplo fue importante para entender cómo se podía manejar la interacción con múltiples clientes sin bloquear el servidor (en el medias5 se manejaba de forma bloqueante). La idea central era guardar el estado de cada cliente y utilizar un selector para multiplexar la entrada y salida con un único thread. De esta forma, cada conexión

podía avanzar según estuviera lista para leer o escribir, sin frenar al resto de las conexiones activas.

Con esa base se avanzó hacia la implementación del proxy SOCKS5. Primero se incorporó la infraestructura no bloqueante provista por la cátedra, como el selector, los buffers y la máquina de estados. Luego se implementó progresivamente el flujo principal del protocolo: aceptar conexiones, negociar el método, autenticar usuarios cuando correspondía, leer el request, resolver o conectar contra el destino y finalmente el intercambio de datos entre cliente y servidor remoto.

3.2 Roles

El trabajo se dividió en tres roles principales. Una parte del grupo se enfocó en el core SOCKS5, otra en el protocolo de monitoreo y otra en aspectos más globales del proyecto, como integración, módulos compartidos y desarrollo del cliente. Esta división permitió avanzar en paralelo, aunque también exigió definir contratos claros entre módulos para evitar dependencias desordenadas. A pesar de esta división de trabajo, todos los integrantes del grupo hacían reuniones regularmente, en donde se explicaban los avances realizados y se realizaba el merge del código en conjunto, logrando que todos comprendieran la estructura del proyecto.

3.3 Problemas encontrados

Uno de los problemas encontrados fue justamente la integración entre las distintas partes del proyecto. Al estar el trabajo dividido entre SOCKS5, monitoreo y módulos transversales, fue necesario decidir bien de que se encargaba cada parte. Hubo casos en el que había lógica de monitoreo en archivos que no correspondían a dicho protocolo.

Otro problema puntual apareció con el comando GET-LOG del protocolo de monitoreo. Inicialmente el registro de accesos estaba orientado principalmente a escribir en stdout, pero el protocolo de monitoreo necesitaba poder consultarlo. Para resolverlo, se agregó un buffer circular en memoria y una interfaz de lectura, separando el módulo que registra accesos del módulo que los consulta.

El manejo de errores en sockets no bloqueantes, también representó un desafío. En una primera versión, el código solo analizaba el valor de retorno de `recv()` y `send()`. Si alguna de estas funciones devolvía `-1`, se interpretaba como un error definitivo o como un cierre de conexión. El problema era que no se estaba consultando `errno`, la variable global donde el sistema operativo indica la causa concreta del error. Empezamos a investigar este error, dado que con muchas conexiones concurrentes había cortes inesperados.

En conclusión, los problemas encontrados fueron principalmente de integración, diseño incremental y detalles propios de la programación no bloqueante. No representaron bloqueos graves, y el uso de inteligencia artificial ayudó a resolverlos con

mayor rapidez, especialmente al momento de comprender ejemplos, validar enfoques y detectar posibles errores en la lógica de sockets y estados.

4 - Limitaciones de la aplicación

La primera limitación importante está relacionada con la cantidad máxima de conexiones concurrentes. El servidor utiliza `pselect()` como mecanismo de multiplexación, por lo que queda limitado por `FD_SETSIZE`, que típicamente es de 1024 file descriptors. Como cada conexión SOCKS5 utiliza dos descriptores, uno para el cliente y otro para el servidor destino, el máximo práctico medido fue de aproximadamente 507 conexiones simultáneas (véase sección 7 - Ejemplos de prueba).

También existe una limitación frente a clientes lentos o conexiones inactivas. Si un cliente mantiene una conexión abierta pero avanza muy lentamente, puede ocupar recursos del servidor durante mucho tiempo. Como no se implementó un timeout de inactividad, un conjunto grande de clientes lentos podría agotar las conexiones disponibles. El modelo no bloqueante evita que esos clientes frenen al resto, pero no impide que consuman descriptores indefinidamente.

Otra limitación es la persistencia del estado. Las métricas, los usuarios configurados en runtime y el registro de accesos se mantienen en memoria, por lo que se pierden al reiniciar el servidor.

Por último, el protocolo de monitoreo utiliza un modelo secuencial de petición y respuesta. No está pensado para implementar pipelining real de comandos, es decir, para que un cliente envíe varias peticiones sin esperar la respuesta anterior. Esto condiciona la experiencia en escenarios con alta latencia o con una gran cantidad de comandos.

5 - Posibles extensiones

Una posible extensión sería reemplazar el uso de `pselect()` por una alternativa más escalable, como `epoll` en Linux o `kqueue` en macOS. Esto permitiría superar el límite impuesto por `FD_SETSIZE` y manejar una cantidad mayor de conexiones concurrentes. Además bajaría el overhead ya que `epoll` devuelve la lista con los fds listos, evitando tener que recorrer todos los fds registrados como se hace ahora mismo.

También sería conveniente agregar timeouts de inactividad. De esta manera, el servidor podría cerrar automáticamente conexiones que permanecen abiertas sin enviar ni recibir datos durante demasiado tiempo. Esto permitiría liberar recursos ocupados por clientes lentos o inactivos y reducir el riesgo de que muchas conexiones de este tipo agoten los descriptores disponibles.

Otra extensión natural sería agregar persistencia para usuarios, métricas y registro de accesos, por ejemplo guardándolos en archivos CSV. Esto permitiría

conservar usuarios creados en runtime, mantener historial de accesos y recuperar información útil luego de reinicios.

Además, se podría incorporar cifrado al canal de monitoreo, por ejemplo mediante TLS. Si se quisiera administrar el servidor de forma remota sería conveniente proteger el token y los comandos enviados.

Por último, el protocolo de monitoreo podría extenderse para soportar pipelining real de comandos. Como mencionamos en la sección anterior, mejoraría la eficiencia en escenarios de mayor latencia o de uso más intensivo del canal administrativo.

6 - Conclusiones

El desarrollo de este proyecto nos permitió comprender de una manera mucho más concreta cómo funciona un servidor capaz de atender una gran cantidad de conexiones concurrentes sin necesitar un hilo por cada cliente. A lo largo de la implementación aprendimos a trabajar con entrada y salida no bloqueante, selectores, buffers y máquinas de estados, entendiendo que cada conexión puede avanzar de forma independiente únicamente cuando está lista para leer o escribir. Esto permitió que, con un solo hilo principal, el servidor pudiera administrar simultáneamente el proxy SOCKS5 y el protocolo de monitoreo, además de mantener controlado el uso de memoria y evitar que una conexión lenta frenara al resto.

7 - Ejemplos de prueba

Para mostrar el funcionamiento se creó un entorno con tres terminales en paralelo: una para el servidor, otra para un destino HTTP local y una tercera para lanzar los clientes. Primero se levanta el servidor, el cual queda escuchando en dos puertos a la vez: el 1080 para el proxy SOCKS5 y el 8080 para el canal para el monitoreo. Así, cargamos un usuario (soy_yo/1234), y el token de administración (tikitaka)

```
negrito@MacBook-Pro-de-Francisco TPE-Protos % ./bin/server -p 1080 -P 8080 -u soy_yo:1234 -t tikitaka
Servidor SOCKSv5 escuchando en el puerto TCP 1080
Servicio de monitoreo escuchando en el puerto TCP 8080
```

En la segunda terminal se levanta un servidor HTTP común en el puerto 9000, que va a hacer de destino de las conexión que pasen por el proxy.

```
negrito@MacBook-Pro-de-Francisco TPE-Protos % python3 -m http.server 9000 --bind 127.0.0.1
Serving HTTP on 127.0.0.1 port 9000 (http://127.0.0.1:9000/) ...
```

Se verifica que el proxy exija las credenciales correctas. Si se intenta una conexión con la contraseña equivocada (1235 en vez de 1234), el proxy rechaza el pedido durante el handshake, y curl corta con el error 97 sin llegar a destino:


```
negrito@MacBook-Pro-de-Francisco TPE-Protos % curl -v -x socks5://soy_yo:1235@127.0.0.1:1080 http://127.0.0.1:9000/
* Trying 127.0.0.1:1080...
* Connected to 127.0.0.1 (127.0.0.1) port 1080
* User was rejected by the SOCKS5 server (1 1).
* Closing connection
curl: (97) User was rejected by the SOCKS5 server (1 1).
```

Con la contraseña correcta, en cambio, el handshake se completa, y el proxy responde “request granted”:

```
negrito@MacBook-Pro-de-Francisco TPE-Protos % curl -v -x socks5://soy_yo:1234@127.0.0.1:1080 http://127.0.0.1:9000/
* Trying 127.0.0.1:1080...
* Connected to 127.0.0.1 (127.0.0.1) port 1080
* SOCKS5 connect to 127.0.0.1:9000 (locally resolved)
* SOCKS5 request granted.
* Connected to 127.0.0.1 (127.0.0.1) port 1080
> GET / HTTP/1.1
> Host: 127.0.0.1:9000
> User-Agent: curl/8.7.1
> Accept: */*
```

Hasta acá el destino era un IP literal. Ahora se prueba que el proxy sepa resolver nombres de dominio. Usando socks5h, la “h” hace que sea el proxy, y no curl, el que resuelve el nombre. Pedimos [example.org](#): el proxy resuelve el dominio, se conecta y el pedido es exitoso también:

```
negrito@MacBook-Pro-de-Francisco TPE-Protos % curl -v -x socks5h://soy_yo:1234@127.0.0.1:1080 http://example.org/
* Trying 127.0.0.1:1080...
* Connected to 127.0.0.1 (127.0.0.1) port 1080
* SOCKS5 connect to example.org:80 (remotely resolved)
* SOCKS5 request granted.
* Connected to 127.0.0.1 (127.0.0.1) port 1080
> GET / HTTP/1.1
> Host: example.org
> User-Agent: curl/8.7.1
> Accept: */*
```

Para ver cómo responde ante la carga, medimos el throughput lanzando descargas en paralelo de un archivo grande mediante un script de bash básico. Para cada nivel de concurrencia (1, 10, 50 y 100 conexiones) mandamos todas las descargas juntas y sumas la velocidad que reporta cada curl. Este es el comando corriendo para el caso de las 10 conexiones:

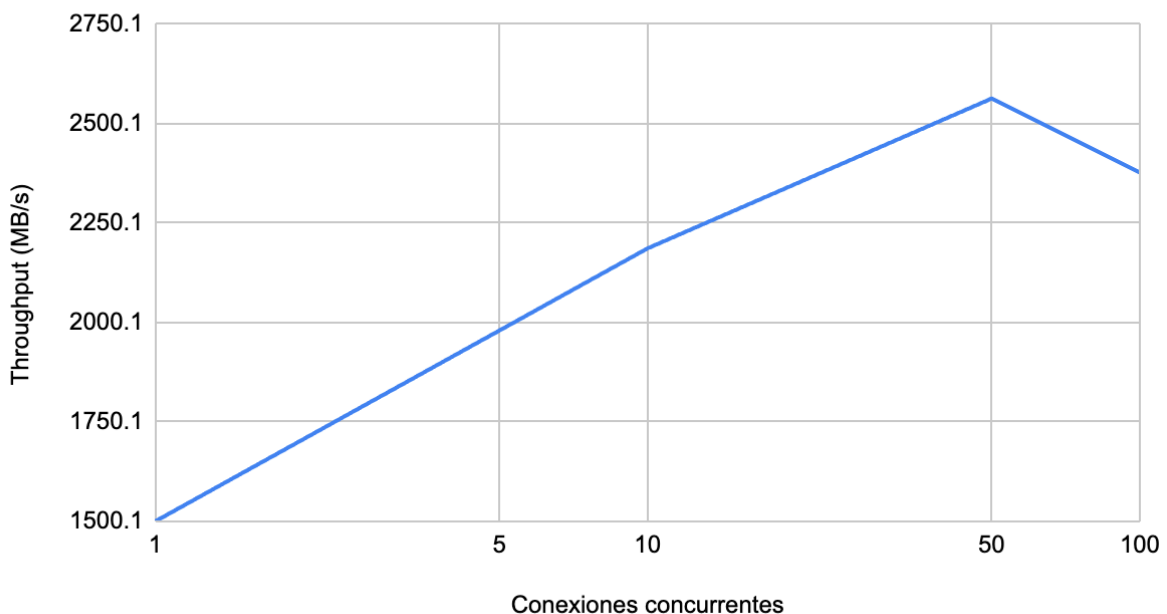
```
negrito@MacBook-Pro-de-Francisco TPE-Protos % { for i in $(seq 1 10); do curl -s -w '%{speed_download}\n' -x socks5h://soy_yo:1234@127.0.0.1:1080 http://127.0.0.1:9000/big.bin -o /dev/null & done; wait; } | awk '{s+=$1;n++} END{printf " %d conex | %8.1f MB/s agregado | %6.1f MB/s por c\n", n, s/1e6, (s/n)/1e6}' | tee -a /tmp/throughput.txt
10 conex | 2186.6 MB/s agregado | 218.7 MB/s por conex
```

Juntando las cuatro corridas leemos el archivo throughput.txt que junto todos los resultados, generando la siguiente tabla.

```
negrito@MacBook-Pro-de-Francisco TPE-Protos % cat /tmp/throughput.txt
== Throughput SOCKS5 (macOS nativo, loopback, big.bin) ==
  1 conex | 0.09s | 1122.0 MB/s agregado | 1122.0 MB/s por conex
 10 conex | 0.49s | 2142.9 MB/s agregado | 214.3 MB/s por conex
 50 conex | 2.16s | 2431.6 MB/s agregado | 48.6 MB/s por conex
100 conex | 4.57s | 2294.8 MB/s agregado | 22.9 MB/s por conex
```

Y graficando el throughput agregado contra la cantidad de conexiones se puede analizar fácilmente el comportamiento:

Throughput del proxy SOCKS5 vs concurrencia



Se puede ver que el throughput agregado sube al principio, al ser no bloqueante y multiplexado, mientras que una conexión espera I/O el servidor aprovecha para atender otras, hace pico alrededor de las 50 conexiones y después baja un poco, porque el overhead de manejar tantas conexiones en un único hilo empieza a pesar. Al mismo tiempo el throughput por conexión se desploma: como todo corre en un solo hilo, cuantas mas conexiones compiten, menos le toca a cada uno.

Ahora analicemos cual es la máxima cantidad de conexiones que acepta el proxy. Para ello creamos un script de python que abre conexiones SOCKS5 completas contra el proxy (handshake + authentication + connect) y las mantiene todas abiertas simultáneamente, agregándole una a una hasta que el proxy no acepta más, e informa cuántas logró sostener a la vez

```
negrito@MacBook-Pro-de-Francisco www % python3 /tmp/maxconns.py  
maximo de conexiones SOCKS simultaneas: 507
```

Como se ve en la imagen anterior se supera el mínimo de 500 conexiones en simultáneo requerido.

8 - Guia de instalación

El primer comando a correr para llevar a cabo la instalación es el de clonado del repositorio el cual puede ser:

```
git clone git@github.com:franciscocostaa/TPE-Protos.git
```

o,

git clone <https://github.com/franciscocostaa/TPE-Protos.git>

Luego, para construir los ejecutables de la aplicación y del servidor se debe utilizar el comando **make**, el cual genera dos binarios en el directorio bin/: el servidor (bin/server) que levanta el proxy SOCKS5 junto con el servicio de monitoreo, y el cliente de administración (bin/client).

Ya compilado, el sistema se pone en marcha ejecutando:

```
./bin/server
```

```
./bin/client
```

9 - Instrucciones para la configuración

Comando	Acción
-p <puerto>	El puerto donde el proxy SOCKS5 espera a los clientes
-l <dirección>	En qué interfaz atiende el proxy
-P <puerto>	El puerto del canal de administracion, separado del proxy
-L <dirección>	En qué interfaz atiende ese canal de administracion
-u <usuario>:<clave>	Da de alta un usuario que puede usar el proxy (límite de 10).
-t <token>	La contraseña del canal de administracion
-v	Muestra la versión
-h	Muestra un mensaje de ayuda.

10 - Ejemplos de configuración y monitoreo

A continuación se llevará a cabo una demostración de nuestros comandos de configuración y monitoreo. Todos los comandos se ejecutan contra el canal de administración, autenticándose con el famoso token: tikitaka.

```
negrito@MacBook-Pro-de-Francisco TPE-Protos % curl -s -o /dev/null -w "curl => %{http_code}\n" -x socks5://soy_yo:1234@127.0.0.1:1080 http://127.0.0.1:9000/
curl => 200
negrito@MacBook-Pro-de-Francisco TPE-Protos % ./bin/client 127.0.0.1 8080 -t tikitaka metrics
Métricas del servidor:
  Conexiones totales      1
  Conexiones concurrentes 0
  Bytes transferidos      567
negrito@MacBook-Pro-de-Francisco TPE-Protos % ./bin/client 127.0.0.1 8080 -t tikitaka log
Accesos registrados (1):
  2026-07-13T19:22:26Z  usuario=soy_yo  :ffff:127.0.0.1:49489 -> 127.0.0.1:9000 (rep 0)
```

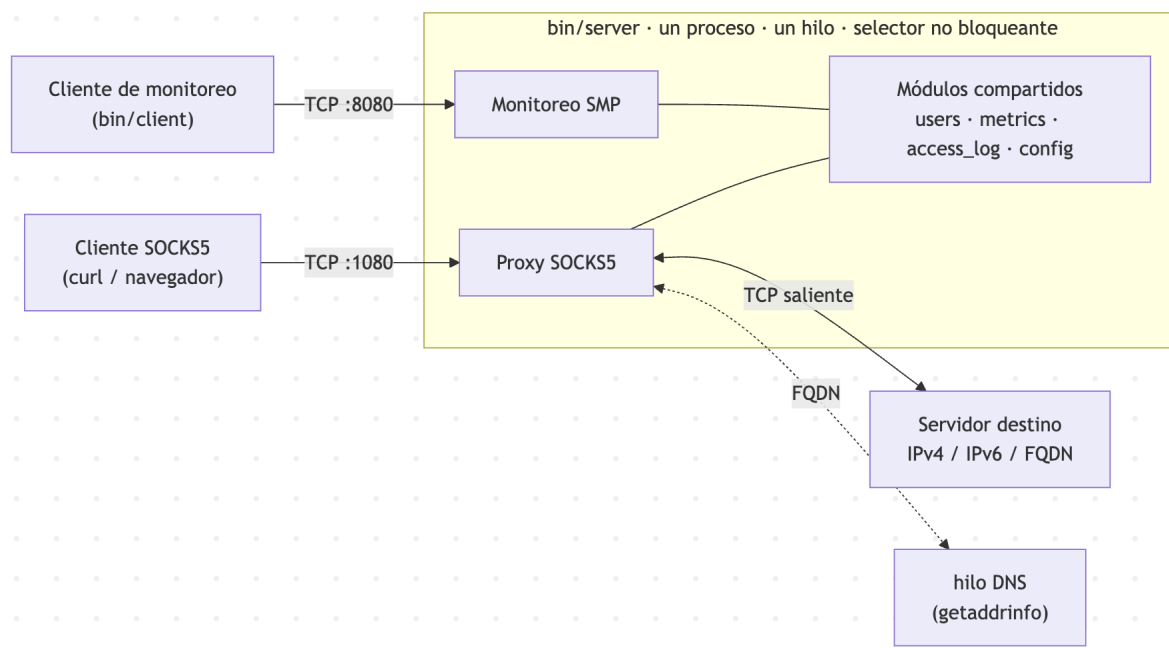
Después mostramos la creación y posterior eliminación de un usuario. Partimos de un único usuario (soy_yo) y luego, con add-user, agregamos a pablito, y la lista pasa a dos. Para comprobar que el alta tiene efecto inmediato, usamos el proxy con las credenciales de pablito y obtenemos un 200. Finalmente lo eliminamos mediante del-user y la lista vuelve a un usuario

```
negrito@MacBook-Pro-de-Francisco TPE-Protos % ./bin/client 127.0.0.1 8080 -t tikitaka users
Usuarios (1):
  soy_yo
negrito@MacBook-Pro-de-Francisco TPE-Protos % ./bin/client 127.0.0.1 8080 -t tikitaka add-user pablito pass1234
Usuario 'pablito' agregado.
negrito@MacBook-Pro-de-Francisco TPE-Protos % ./bin/client 127.0.0.1 8080 -t tikitaka users
Usuarios (2):
  soy_yo
  pablito
negrito@MacBook-Pro-de-Francisco TPE-Protos % curl -s -o /dev/null -w "pablito => %{http_code}\n" -x socks5://pablito:pass1234@127.0.0.1:1080 http://127.0.0.1:9000/
pablito => 200
negrito@MacBook-Pro-de-Francisco TPE-Protos % ./bin/client 127.0.0.1 8080 -t tikitaka del-user pablito
Usuario 'pablito' eliminado.
negrito@MacBook-Pro-de-Francisco TPE-Protos % ./bin/client 127.0.0.1 8080 -t tikitaka users
Usuarios (1):
  soy_yo
```

Por último, el canal de administración exige su propio token, si utilizamos uno equivocado (takatiki en este caso), o si directamente no ingresamos uno, se muestran los mensajes apropiados:

```
negrito@MacBook-Pro-de-Francisco TPE-Protos % ./bin/client 127.0.0.1 8080 -t takatiki metrics
client: autenticación fallida
negrito@MacBook-Pro-de-Francisco TPE-Protos % ./bin/client 127.0.0.1 8080 metrics
client: el comando 'metrics' requiere autenticación: pasá -t <token>
negrito@MacBook-Pro-de-Francisco TPE-Protos % ./bin/client 127.0.0.1 8080 -t tikitaka metrics
Métricas del servidor:
  Conexiones totales      4
  Conexiones concurrentes 0
  Bytes transferidos      1701
```

11 - Documento del diseño del proyecto



Como muestra el diagrama, todo el sistema corre en un único proceso y un único hilo, que multiplexa la entrada/salida de forma no bloqueante mediante un selector. Dentro de ese proceso hay dos servicios independientes escuchando en puertos distintos: el proxy SOCKS5 (puerto 1080), que recibe a los clientes y reenvía su tráfico al servidor destino, y el canal de monitoreo SMP (puerto 8080), que atiende al cliente de administración. Ambos se apoyan en un conjunto de módulos compartidos - usuarios, métricas, registro de accesos y configuración-, de modo que tanto el proxy como el canal de monitoreo leen y modifican el mismo estado (por ejemplo, un usuario dado de alta por el administrador queda disponible al instante para el proxy). Cada conexión se modela como una máquina de estados propia, lo que permite atender cientos de clientes a la vez sin dedicarle un hilo a cada uno.

La decisión central de diseño es, justamente, ese modelo de un solo hilo con E/S no bloqueante: en lugar de bloquearse esperando a un cliente lento, el servidor avanza cada conexión solo cuando su socket está listo para leer o escribir, y usa buffers de tamaño fijo con control de flujo para no acumular memoria sin límite al reenviar los datos. La única excepción a la regla de un solo hilo es la resolución de nombres (getaddrinfo), que es bloqueante y por eso se ejecuta en un hilo aparte; ese hilo no toca ningún socket y solo le avisa al hilo principal cuando terminó de resolver, para que este continúe la conexión. El resultado es un servidor concurrente, con memoria acotada y robusto ante conexiones lentas o destinos caídos.